

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

**COURSE NAME: Database Management Systems
COURSE CODE: CSA05**

1. Functional Dependencies and Candidate Keys

Given relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Functional Dependencies

1. $SID \rightarrow SName$
A Student ID uniquely determines the Student Name.
2. $CID \rightarrow CName, Instructor$
A Course ID uniquely determines the Course Name and Instructor.
3. $(SID, CID) \rightarrow Grade$
The combination of Student ID and Course ID determines the Grade obtained.

Therefore:

$SID \rightarrow SName$

$CID \rightarrow CName, Instructor$

$SID, CID \rightarrow Grade$

Candidate Key

Compute the closure:

$\{SID, CID\}^+$

$= \{SID, CID\}$

Using $SID \rightarrow SName$:

$= \{SID, CID, SName\}$

Using $CID \rightarrow CName, Instructor$:

$= \{SID, CID, SName, CName, Instructor\}$

Using $SID, CID \rightarrow Grade$:

$= \{SID, CID, SName, CID, CName, Instructor, Grade\}$

Thus, $\{SID, CID\}^+$ contains all attributes.

Answer

Candidate Key = {SID, CID}

Neither SID nor CID alone determines all attributes. Therefore, (SID, CID) is the minimal candidate key.

2. Conversion of an Unnormalized Relation into 1NF

Consider the unnormalized relation:

STUDENT(SID, SName, Courses)

Example:

SID	SName	Courses
S10 1	Arun	DBMS, Java, Python
S10 2	Priya	DBMS, C++

The Courses attribute contains multiple values. Therefore, the relation is in UNF (Unnormalized Form).

Step 1: Identify Repeating Groups

The attribute Courses contains multiple values for one student. This violates the atomicity requirement of 1NF.

Step 2: Make Each Attribute Atomic

Split each multivalued course into separate tuples.

1NF Relation

SID	SName	Course
S10 1	Arun	DBMS
S10 1	Arun	Java

S10 1	Arun	Python
S10 2	Priya	DBMS
S10 2	Priya	C++

Justification

A relation is in First Normal Form (1NF) when:

- Every attribute contains only atomic values.
- There are no repeating groups.
- Each row-column intersection contains a single value.
- Each tuple can be uniquely identified.

Result

The relation has been successfully converted from UNF to 1NF because the multivalued attribute Courses has been converted into atomic values.

3. Decomposition into 2NF

Given:

$R(A, B, C, D, E)$

Functional Dependencies:

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Step 1: Find Candidate Key

Compute:

A^+

$A \rightarrow BC$

Therefore:

$A^+ = \{A, B, C\}$

$BC \rightarrow D$:

$A^+ = \{A, B, C, D\}$

$D \rightarrow E$:

$A^+ = \{A, B, C, D, E\}$

Therefore:

Candidate Key = A

Step 2: Check for Partial Dependencies

A partial dependency occurs when a non-prime attribute depends on only part of a composite candidate key.

Here, the candidate key is A, which contains only one attribute.

Therefore, there can be no partial dependency.

Step 3: 2NF Decomposition

Since there are no partial dependencies:

$R(A, B, C, D, E)$ is already in 2NF.

Result

Candidate Key: A

Partial Dependencies: None

2NF Decomposition: No decomposition is required.

The relation remains:

$R(A, B, C, D, E)$

4. 3NF Decomposition

Given relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional Dependencies:

1. EmpID $\rightarrow$ EmpName, DeptID
2. DeptID $\rightarrow$ DeptName, ManagerID

Step 1: Identify Candidate Key

Since EmpID determines all other attributes:

EmpID $\rightarrow$ EmpName, DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

Candidate Key = EmpID

Step 2: Identify Transitive Dependency

We have:

EmpID $\rightarrow$ DeptID

and

DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptName, ManagerID

through DeptID.

This is a transitive dependency.

Step 3: Decompose into 3NF

Create the following relations:

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Functional Dependency:

EmpID $\rightarrow$ EmpName, DeptID

Primary Key: EmpID

DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Functional Dependency:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Primary Key: DeptID

Result

The transitive dependency has been removed.

Final 3NF Relations:

- EMPLOYEE(EmpID, EmpName, DeptID)
- DEPARTMENT(DeptID, DeptName, ManagerID)

Thus, the database is normalized to Third Normal Form (3NF).

5. BCNF Verification and Decomposition

Given:

$R(A, B, C, D)$

Functional Dependencies:

- $AB \rightarrow C$
- $C \rightarrow D$
- $D \rightarrow A$

Step 1: Find Candidate Keys

Compute:

$(AB)^+$

$AB \rightarrow C$ gives $\{A, B, C\}$

$C \rightarrow D$ gives $\{A, B, C, D\}$

Therefore:

AB is a candidate key.

Now consider BC:

$C \rightarrow D$ and $D \rightarrow A$

Therefore:

$BC^+ = \{B, C, D, A\}$

Hence BC is also a candidate key.

Similarly:

BD^+

$D \rightarrow A$, and $AB \rightarrow C$

Therefore:

$BD^+ = \{A, B, C, D\}$

Hence BD is also a candidate key.

Candidate Keys

AB, BC, BD

Step 2: Check BCNF

A relation is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a superkey.

Check:

- $AB \rightarrow C$: AB is a superkey ✓
- $C \rightarrow D$: C is not a superkey ✗
- $D \rightarrow A$: D is not a superkey ✗

Therefore, R is not in BCNF.

Step 3: Decompose

Using $C \rightarrow D$:

$R_1(C, D)$

$R_2(A, B, C)$

R1 is in BCNF because C is the key.

For R2:

$AB \rightarrow C$

AB is the key of R2.

Therefore, R2 is also in BCNF.

Final BCNF Decomposition

- R1(C, D)
- R2(A, B, C)

Result

The original relation is not in BCNF because $C \rightarrow D$ violates BCNF. The decomposition above produces BCNF relations.

6. Lossless-Join Decomposition Using Chase Algorithm

Consider:

R(A, B, C)

Functional Dependency:

$A \rightarrow B$

Decompose R into:

- R1(A, B)
- R2(A, C)

Step 1: Create Initial Tableau

Relation	A	B	C
R1	a 1	a2	b1
R2	a 1	b2	a3

The common attribute is A.

Step 2: Apply Functional Dependency $A \rightarrow B$

Since both rows have the same value for A, their B values must become equal.

After applying $A \rightarrow B$:

Relation	A	B	C
R1	a	a	b1
	1	2	
R2	a	a	a3
	1	2	

Step 3: Check for Lossless Join

The decomposition satisfies the lossless join condition because the common attribute A functionally determines all attributes of R1:

$A \rightarrow B$

Therefore:

$R1 \cap R2 = \{A\}$

and

$A \rightarrow R1$

Hence, the decomposition is lossless.

Result

$R(A,B,C) \rightarrow R1(A,B), R2(A,C)$ is a lossless-join decomposition.

Joining R1 and R2 reconstructs the original relation without generating spurious tuples.

7. Multivalued Dependencies and 4NF

Given:

TEACHER(TID, Subject, Hobby)

Assume a teacher can independently teach multiple subjects and have multiple hobbies.

Multivalued Dependencies

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

This means that the subjects and hobbies of a teacher are independent of each other.

Example:

TID	Subject	Hobby
T01	DBMS	Reading
T01	DBMS	Cricket
T01	Java	Reading
T01	Java	Cricket

The combinations are repeated unnecessarily.

4NF Decomposition

Decompose the relation into:

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

TID	Subject
T01	DBMS
T01	Java

TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
-----	-------

T01	Reading
T01	Cricket

Result

The multivalued dependencies are separated into independent relations:

- TEACHER_SUBJECT(TID, Subject)
- TEACHER_HOBBY(TID, Hobby)

Both relations are in Fourth Normal Form (4NF).

This removes redundancy caused by independent multivalued attributes.

8. Join Dependencies and 5NF

Consider the relation:

SUPPLY(Supplier, Part, Project)

A supplier may supply a part to a project.

Suppose the information contains the following independent relationships:

- Supplier supplies Part.
- Supplier works with Project.
- Part is required for Project.

This can produce redundancy when all three are stored together.

Original Relation

SPJ(Supplier, Part, Project)

Example:

Supplier	Part	Project
S1	P1	J1
S1	P2	J1

S2	P1	J1
S2	P2	J1

If one relationship changes, multiple tuples may need modification.

Join Dependency

The relation can be represented through:

SP(Supplier, Part)

SJ(Supplier, Project)

PJ(Part, Project)

The join dependency is:

$SPJ = SP \bowtie SJ \bowtie PJ$

5NF Decomposition

Decompose SPJ into:

1. SUPPLIER_PART(Supplier, Part)
2. SUPPLIER_PROJECT(Supplier, Project)
3. PART_PROJECT(Part, Project)

Result

The three smaller relations remove redundancy caused by complex join dependencies.

The original relation can be reconstructed using:

$SUPPLIER_PART \bowtie SUPPLIER_PROJECT \bowtie PART_PROJECT$

Therefore, the relations are in Fifth Normal Form (5NF), also called Project-Join Normal Form (PJNF).

9. College Database Anomalies and Normalization to 3NF

Consider the poorly designed relation:

COLLEGE(StudentID, StudentName, DeptID, DeptName, HOD, CourseID, CourseName)

Functional Dependencies:

- $\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}, \text{HOD}$
- $\text{CourseID} \rightarrow \text{CourseName}$

1. Insertion Anomaly

A new department cannot be stored unless at least one student is enrolled in it.

For example, adding a new department AI Department without a student may not be possible.

2. Deletion Anomaly

If the last student in a department is deleted, information about the department and HOD may also be lost.

3. Update Anomaly

If the HOD changes, the HOD information must be updated in every tuple belonging to that department.

Failure to update all rows creates inconsistent data.

Step 1: Identify Candidate Key

If each student takes multiple courses, the composite key is:

$(\text{StudentID}, \text{CourseID})$

Step 2: Identify Dependencies

$\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{HOD}$

$\text{CourseID} \rightarrow \text{CourseName}$

There is a transitive dependency:

$\text{StudentID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{HOD}$

Step 3: Normalize to 3NF

STUDENT

STUDENT(StudentID, StudentName, DeptID)

Primary Key: StudentID

DEPARTMENT

DEPARTMENT(DeptID, DeptName, HOD)

Primary Key: DeptID

COURSE

COURSE(CourseID, CourseName)

Primary Key: CourseID

ENROLLMENT

ENROLLMENT(StudentID, CourseID)

Primary Key: (StudentID, CourseID)

Result

The final database schema is:

- STUDENT(StudentID, StudentName, DeptID)
- DEPARTMENT(DeptID, DeptName, HOD)
- COURSE(CourseID, CourseName)
- ENROLLMENT(StudentID, CourseID)

This design removes insertion, deletion, and update anomalies and achieves 3NF.

10. Attribute Closure and Candidate Keys

Since no specific relation or functional dependencies are provided in Question 10, consider the following example:

Given relation:

R(A, B, C, D, E)

Functional Dependencies:

- $A \rightarrow BC$
- $B \rightarrow D$
- $CD \rightarrow E$

Step 1: Compute A^+

Start with:

$$A^+ = \{A\}$$

Using:

$$A \rightarrow BC$$

$$A^+ = \{A, B, C\}$$

Using:

$$B \rightarrow D$$

$$A^+ = \{A, B, C, D\}$$

Using:

$$CD \rightarrow E$$

Since C and D are available:

$$A^+ = \{A, B, C, D, E\}$$

Therefore:

$$A^+ = \{A, B, C, D, E\}$$

Hence, A is a superkey.

Since no attribute can be removed from the single attribute set A:

A is a candidate key.

Other Closures

$$B^+ = \{B, D\}$$

$$C^+ = \{C\}$$

$$D^+ = \{D\}$$

$$E^+ = \{E\}$$

None of these determine all attributes.

Final Result

$A^+ = \{A, B, C, D, E\}$

Therefore:

Candidate Key = $\{A\}$

The closure method is used to determine whether an attribute set is a superkey and to identify all minimal superkeys, called candidate keys.